

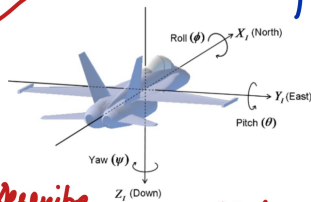
Drone Control Loop

2



* **Throttle**: The current, real-time number the Remote Controller (RC) sends the Flight Controller (FC), comes from joystick.

Let's say range from 0 (motors off) to 1000 (full power)



Describe 3D orientation of object with respect to fixed reference frame



Example:

Drone Flying -> Wind gust -> Unstable! -> PID
To stabilize: Push left up, drop right down

Left motors spin faster, Right slower

So $motor_{left} = throttle + roll\ correction$
 $motor_{right} = throttle - roll\ correction$

Assuming all motors spin in the same direction



Motor PWM

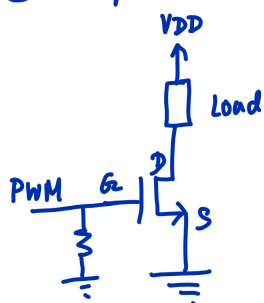
① Using Electronic Speed Controllers (ESCs) -> expect a pulse every 20ms.

* Pulse of 1ms = 0% Throttle
* Pulse of 2ms = 100% Throttle

External ESC -> may add weight!

So, take throttle (0-1000) -> map to pulse width (1ms - 2ms) -> Set PWM pulse

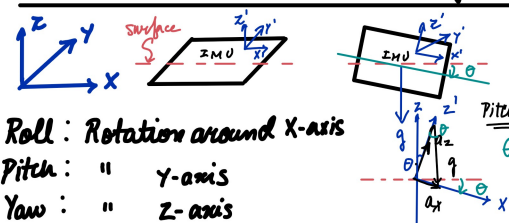
② Simple low side switching (+ flyback diode connected across motor load (inductive) to prevent damage from voltage spikes)



So, can map $motor_{left}$ & $motor_{right}$ directly to PWM percentage

* Note: Choose high PWM frequency to reduce vibrations ($\approx 20\text{ kHz}$)

Calculate Roll and Pitch angles



Roll: Rotation around X-axis
Pitch: " Y-axis
Yaw: " Z-axis

Similarly, Roll:

$$\phi = \tan^{-1} \left(\frac{a_y}{\sqrt{a_x^2 + a_z^2}} \right)$$

$$\theta = \tan^{-1} \left(\frac{a_x}{\sqrt{a_y^2 + a_z^2}} \right)$$

Remote Controller

Inputs



Joystick1



Joystick2

analog-joystick.h
analog-joystick.c

1. Read ADC (1 channel)

calls

decode-joystick.h
decode-joystick.c

2. Decode ADC readings
(Mode 2 config; desired
Throttle, Yaw, Pitch, Roll)

TYPE

Gives Semaphore to wake
ESB

radio-esb.h
radio-esb.c

3. Prepare & TX Data Packet



To prevent data being Read as it's being written to,
use mutex:

Joystick → Lock → Write → Unlock → Give Sem → ESB

Sample ADC @ 100 Hz

12 bit resolution - 0 to 4095

Flight Controller

IMU → Sensor Fusion → PID → Motor PWM

ESB Rx Interrupt

